Project - High Level Design

On

Non-Profit Support Triage Agent

Course Name:Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Uday Patidar	EN23CSL11023
2.	Neha Tamboli	EN22CS301641
3.	Riddhima Taose	EN22CS303042
4.	Rohan Rawat	EN23CS3L1016
5.	Naman Dwivedi	EN22CS301627

Group Name:04D6

Project Number:AAI-15

Industry Mentor Name:

University Mentor Name: Nishant Shrivastav

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The rapid advancement of artificial intelligence technologies has significantly transformed the way organizations manage communication and customer engagement. Non-profit organizations, in particular, face unique challenges in handling large volumes of incoming communication such as donation inquiries, volunteer requests, grievance messages, partnership proposals, and general information requests. Unlike corporate organizations, non-profits often operate with limited staff and resources, making efficient communication management critical for maintaining operational effectiveness and stakeholder trust.

The proposed Non-Profit Support Triage Agent is an intelligent automated communication processing system designed to assist non-profit organizations in prioritizing, classifying, and responding to incoming support messages. The system leverages modern Natural Language Processing (NLP) techniques, Large Language Models (LLMs), and intelligent agent frameworks to automate the initial triage process. By doing so, it reduces the manual workload of support teams and improves response efficiency.

The system analyzes incoming text messages, extracts relevant contextual information, classifies the urgency and intent of the communication, and generates a suitable response draft. This automated pipeline enables organizations to handle high communication volumes efficiently while maintaining response quality and consistency.

The project integrates multiple modern AI technologies including LangChain agent orchestration, vector database storage through ChromaDB, prompt engineering strategies, and an interactive Streamlit-based frontend dashboard. The result is a comprehensive AI-powered triage system capable of enhancing communication workflows in non-profit organizations.

1.1 Scope of the Document

This document provides a comprehensive high-level design of the Non-Profit Support Triage Agent system. It focuses on the architectural design, functional modules, system interactions, and technical considerations required to implement the proposed solution. The document aims to provide clarity regarding system structure, operational flow, and integration of AI-based processing components.

The scope includes an overview of system architecture, component interactions, data management strategies, application interfaces, security considerations, and performance

requirements. It also outlines the integration of external APIs such as language model services and vector databases. The document intentionally avoids low-level implementation details such as specific code logic or algorithmic optimization techniques, as these fall under low-level design documentation.

This HLD document serves as a reference for developers, project evaluators, and stakeholders involved in system development and deployment.

1.2 Intended Audience

The primary audience for this document includes software developers, AI engineers, academic supervisors, project evaluators, and organizational stakeholders who require a technical understanding of the system design. Developers will use this document to understand the modular structure and system interactions before implementing detailed code-level solutions.

Academic evaluators will utilize this document to assess the technical feasibility, architectural clarity, and design justification of the project. Non-profit stakeholders may refer to this document to understand how the system supports communication automation and operational efficiency.

Additionally, this document may serve as a reference for future system enhancements, maintenance activities, and potential enterprise deployment.

1.3 System Overview

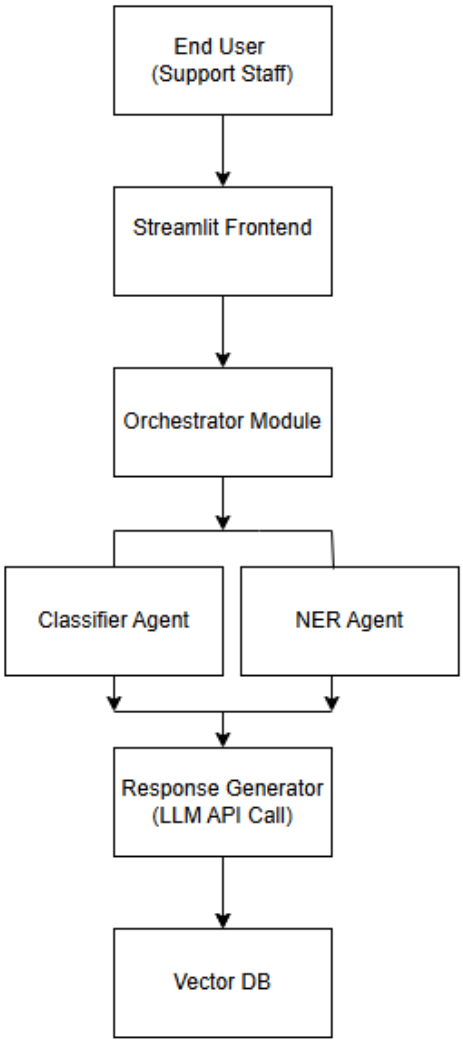
The Non-Profit Support Triage Agent operates as an intelligent middleware system that processes incoming communication messages and converts them into structured, actionable insights. It functions as a communication assistant that automates initial message triage before human intervention.

The system workflow begins when a user submits a communication message through the frontend interface. The system preprocesses the message to remove unnecessary characters, normalize formatting, and prepare the text for AI analysis. Following preprocessing, the classification agent evaluates the message to determine its urgency level and intent category.

Subsequently, the Named Entity Recognition agent extracts structured information such as donor names, email addresses, dates, locations, or monetary amounts. This structured data enhances response personalization and facilitates analytical insights.

The response generation agent then creates a contextually appropriate reply using a large language model. The generated response is displayed on the dashboard for review and can be used directly or modified by human operators.

The system also stores processed data for analytics, reporting, and trend analysis, enabling organizations to monitor communication patterns and improve operational decision-making.



System Architecture

2. System Design

The system design follows a modular architecture to ensure flexibility, scalability, and maintainability. Each component is designed to operate independently while contributing to the overall workflow. This modular approach simplifies future upgrades, debugging, and integration of additional functionalities.

The system is broadly divided into three layers: presentation layer, application logic layer, and data management layer. This layered design ensures separation of concerns, which improves system reliability and ease of maintenance.

2.1 Application Design

The application design emphasizes user accessibility, AI-driven automation, and efficient data processing. The presentation layer is implemented using Streamlit, a lightweight web framework that supports rapid development of interactive dashboards. The frontend provides a user-friendly interface for entering messages, viewing classification results, analyzing extracted entities, and reviewing generated responses.

The application logic layer contains the core AI agents responsible for classification, entity extraction, and response generation. These agents are orchestrated using an orchestrator module that ensures sequential execution of tasks and proper data flow between components.

The data layer handles persistent storage of communication history, embeddings, analytics data, and session information. ChromaDB serves as the vector database to store semantic embeddings of messages, enabling contextual retrieval and similarity analysis.

This design supports efficient scaling, integration with external systems, and adaptability to evolving organizational needs.

2.2 Process Flow

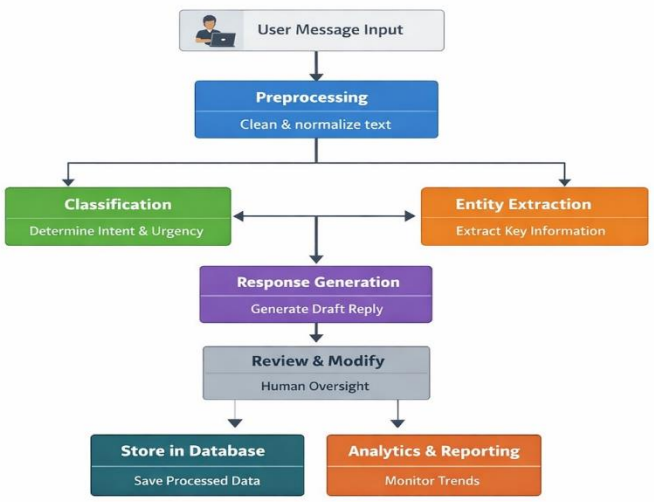
The system process flow is designed to ensure efficient communication processing while maintaining accuracy and reliability. When a user submits a message, preprocessing ensures that the input text is clean and normalized. This step removes irrelevant formatting, corrects spacing issues, and standardizes text input.

Following preprocessing, the classification agent analyzes the message content to determine urgency levels such as high, medium, or low. Simultaneously, it identifies the intent category, such as donation inquiry, volunteer request, complaint, or general inquiry.

The NER agent extracts structured entities from the message, which enhances contextual understanding. These entities may include personal names, donation amounts, dates, or locations.

Finally, the response generation agent creates a draft reply tailored to the identified intent and urgency level. The output is presented on the dashboard and stored for analytics purposes.

This sequential process ensures systematic handling of communication data.



Communication Processing Workflow

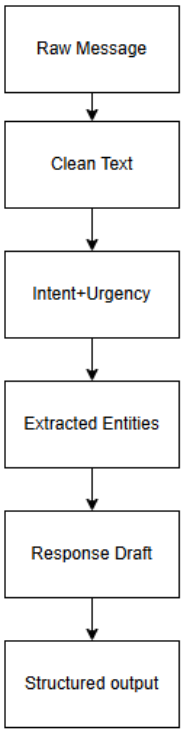
2.3 Information Flow

Information flows through the system in a structured and controlled manner. The raw message initially enters the preprocessing module, where it is transformed into a standardized format suitable for AI analysis. This standardized message then passes through classification and entity extraction agents.

Each agent produces structured outputs that serve as inputs for subsequent processing stages. The classification output informs the response generation agent about urgency and intent, while the entity extraction output provides contextual data for personalized response drafting.

The orchestrator ensures smooth data transfer between agents, preventing information loss and maintaining processing continuity. The final structured output includes classification results, extracted entities, generated responses, and historical storage entries.

Stage	Input	Output
Preprocessing	Raw text	Clean text
Classification	Clean text	Intent/Urgency
Entity Extraction	Clean text	Structured entities
Response Generation	All data	Draft response



Information Flow Design

2.4 Component Design

The frontend component provides an interactive interface that allows users to submit messages and view results. It also includes analytical dashboards that visualize communication trends and operational insights.

The classification agent uses prompt-based LLM processing to categorize messages based on urgency and intent. The NER agent uses NLP techniques to extract structured information from unstructured text. The response generation agent utilizes contextual prompts to generate appropriate replies.

The orchestrator coordinates agent execution, ensuring proper sequencing and error handling. The vector database stores embeddings to support semantic search, contextual retrieval, and analytics.

Each component is designed independently, promoting modularity and ease of maintenance.

2.5 Key Design Considerations

Scalability is a major design consideration, as non-profit organizations may experience fluctuating communication volumes. The system is designed to handle increased message loads without compromising performance.

Accuracy is another critical factor. Prompt engineering techniques are used to improve classification reliability and response relevance.

Security considerations include protecting sensitive donor information and ensuring secure API communication. Data encryption, secure environment variables, and controlled access mechanisms are implemented.

Maintainability is ensured through modular architecture, enabling future updates without major system restructuring.

2.6 API Catalogue

The system integrates several external APIs, primarily large language model APIs for classification, entity extraction, and response generation. Embedding APIs support semantic vector storage within ChromaDB. Additionally, Streamlit APIs facilitate frontend interface rendering.

These APIs are used in a controlled manner to ensure system stability, reliability, and performance efficiency.

API	Purpose
LLM API	Text generation
Embedding API	Vector storage
Streamlit API	UI rendering

3. Data Design

Data design is a critical aspect of the Non-Profit Support Triage Agent, as the system processes large volumes of textual communication that must be structured, stored, and analyzed efficiently. Proper data design ensures accuracy, scalability, privacy protection, and reliable analytics.

The system stores both structured and semi-structured data, including original communication messages, classification outcomes, extracted entities, generated responses, and timestamps. This structured approach allows efficient retrieval, analysis, and reporting.

Additionally, vector embeddings generated through the embedding API are stored in the ChromaDB vector database. These embeddings enable semantic similarity search, contextual retrieval, and historical analytics.

Effective data design ensures that the system remains scalable while maintaining data integrity and accessibility.

3.1 Data Model

The data model of the system is designed to capture communication interactions in a structured format while preserving contextual meaning. Each incoming message is stored along with metadata generated during processing. This includes classification labels such as urgency level and intent category, extracted named entities, generated responses, and timestamps indicating when processing occurred.

This structured storage enables efficient historical analysis and reporting. For example, organizations can identify patterns such as increased donation inquiries during fundraising campaigns or spikes in complaints following specific events.

The vector database component stores semantic embeddings of communication messages. These embeddings allow similarity-based retrieval, which improves context-aware response generation and supports advanced analytics.

The data model is designed to be flexible so that additional attributes can be incorporated in future system updates without significant restructuring.

Field Name	Type	Description
message_id	UUID	Unique ID
original_message	Text	User message
urgency	String	High/Medium/Low
intent	String	Donation/Volunteer/etc
extracted_entities	JSON	Structured data
generated_response	Text	Draft reply
timestamp	Datetime	Processing time
embedding_vector	Vector	Stored in ChromaDB

3.2 Data Access Mechanism

Data within the system is accessed through multiple mechanisms designed to ensure efficiency, scalability, and security. The primary method of data access involves API calls to external services such as the LLM API and embedding API. These APIs process textual data and return structured outputs that are stored within the system.

The vector database provides semantic search capabilities by allowing retrieval of previously processed messages based on contextual similarity. This enhances response generation accuracy and supports analytical insights.

Additionally, session state management within the Streamlit framework ensures temporary storage of user interactions during runtime. This mechanism enables real-time analytics, history tracking, and dashboard visualization without requiring persistent database queries for every interaction.

This multi-layered data access strategy balances performance efficiency with data reliability.

3.3 Data Retention Policies

Data retention policies are essential to ensure privacy compliance, security, and efficient storage management. The system adopts a layered retention strategy where short-term session data is temporarily stored for active interactions, while long-term communication data is archived securely for analytics and reporting purposes.

Sensitive donor or volunteer information is handled with strict confidentiality. Access controls and encryption techniques are implemented to prevent unauthorized data exposure. Data anonymization techniques may be applied when storing analytics datasets to protect personal information.

Retention policies also consider regulatory compliance requirements related to data protection, ensuring that sensitive communication data is not retained longer than necessary.

These policies support both operational efficiency and ethical data management practices.

3.4 Data Migration

The system is designed with future scalability in mind, allowing smooth data migration to enterprise-grade platforms if required. Potential migration options include cloud-based database services, integration with customer relationship management (CRM) systems used by non-profit organizations, and interoperability with external analytics platforms.

Cloud migration would enable improved scalability, backup reliability, and distributed access. CRM integration would allow seamless communication management within existing organizational workflows.

The modular architecture ensures that data migration can be performed with minimal disruption to system functionality.

4. Interfaces

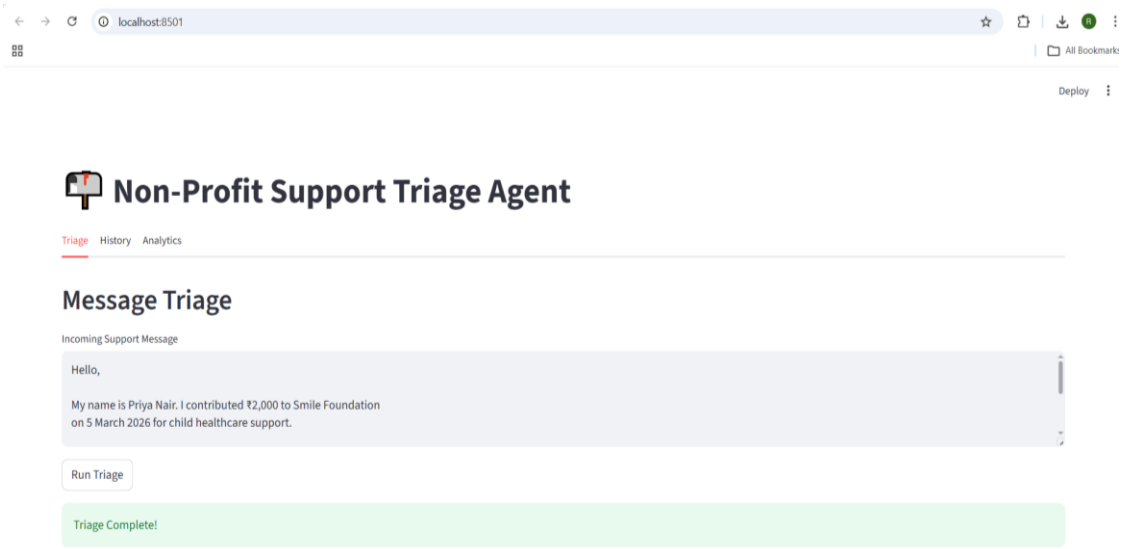
Interfaces play a critical role in ensuring smooth interaction between users, AI agents, and external systems. The primary user interface is implemented using Streamlit, which provides an intuitive web-based dashboard for message input, response visualization, analytics monitoring, and communication history tracking.

The user interface is designed with simplicity and accessibility in mind. Users can submit messages, view classification outcomes, review extracted entities, and analyze generated responses without requiring technical expertise.

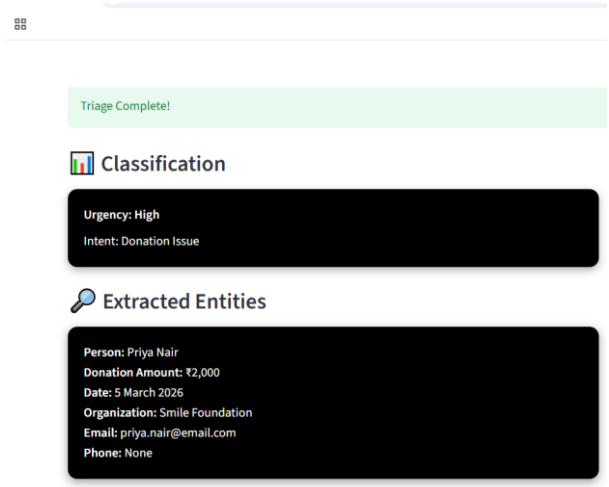
System interfaces include integration with LLM APIs, embedding APIs, and the ChromaDB vector database. These interfaces enable seamless communication between system components and external AI services.

Streamlit UI Screenshot showing:

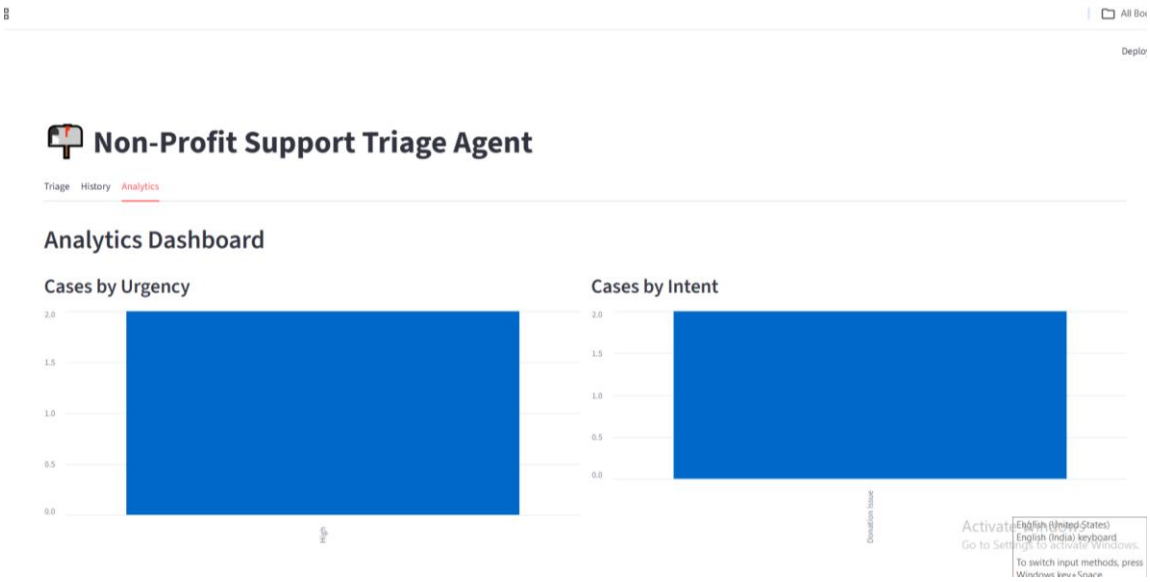
- Message input box



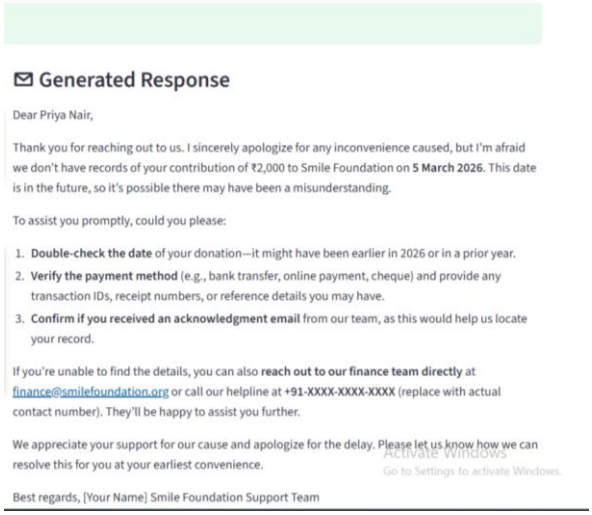
- Classification result box



- Analytics charts



- Response display



Generated Response

Dear Priya Nair,

Thank you for reaching out to us. I sincerely apologize for any inconvenience caused, but I'm afraid we don't have records of your contribution of ₹2,000 to Smile Foundation on **5 March 2026**. This date is in the future, so it's possible there may have been a misunderstanding.

To assist you promptly, could you please:

1. **Double-check the date** of your donation—it might have been earlier in 2026 or in a prior year.
2. **Verify the payment method** (e.g., bank transfer, online payment, cheque) and provide any transaction IDs, receipt numbers, or reference details you may have.
3. **Confirm if you received an acknowledgment email** from our team, as this would help us locate your record.

If you're unable to find the details, you can also **reach out to our finance team directly** at finance@smilefoundation.org or call our helpline at **+91-XXXX-XXXX-XXXX** (replace with actual contact number). They'll be happy to assist you further.

We appreciate your support for our cause and apologize for the delay. Please let us know how we can resolve this for you at your earliest convenience.

Best regards, [Your Name] Smile Foundation Support Team

5. State and Session Management

State management ensures continuity of user interactions during system operation. The system uses Streamlit session state to store temporary data such as message history, classification results, analytics information, and generated responses.

This approach prevents loss of information during active sessions and improves user experience by maintaining continuity between interactions. Session state also supports analytics dashboards by enabling real-time aggregation of communication data.

Proper session management ensures system responsiveness and reliability while minimizing unnecessary database queries.

6. Caching

Caching mechanisms significantly improve system performance by reducing redundant computations and API calls. The system leverages vector embedding storage within ChromaDB as a caching mechanism for semantic representations of communication messages.

By storing embeddings, the system can quickly retrieve contextual information without repeatedly calling external APIs. This reduces response latency and operational costs.

Caching also improves scalability by allowing the system to handle larger communication volumes without performance degradation.

Effective caching strategies are essential for maintaining system efficiency in AI-driven applications.

7. Non-Functional Requirements

Non-functional requirements define the operational quality attributes of the system, including security, performance, reliability, and usability. These requirements ensure that the system performs efficiently under real-world conditions while maintaining data integrity and user trust.

7.1 Security Aspects

Security is a critical concern when handling communication data, especially in non-profit organizations where donor information may be sensitive. The system implements secure API key management through environment variables, ensuring that credentials are not exposed within application code.

Encryption techniques are used to secure communication between system components and external APIs. Access control mechanisms prevent unauthorized data access.

Data anonymization techniques may be applied when generating analytics reports to protect individual identities. Regular security audits and monitoring further enhance system security.

These measures ensure that sensitive communication data remains protected.

7.2 Performance Aspects

Performance optimization is essential for ensuring timely response generation and efficient communication processing. The system leverages fast LLM inference, caching mechanisms, and modular processing architecture to achieve optimal performance.

Efficient caching reduces redundant API calls, while modular agent design ensures that individual components can be optimized independently. Streamlit frontend optimization enhances dashboard responsiveness.

The expected performance benefits include reduced response latency, improved classification accuracy, enhanced user experience, and reduced manual workload for organizational staff.

8. References

1.Lewis, M., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., et al. (2020).
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
Advances in Neural Information Processing Systems (NeurIPS).

2.OpenAI. (2024).
OpenAI API Documentation.
Available at: https://platform.openai.com/docs

3. LangChain Documentation. (2024).
LangChain Framework for LLM Applications.
Available at: https://docs.langchain.com

4. Streamlit Inc. (2024).
Streamlit Documentation – Building Data Apps in Python.
Available at: https://docs.streamlit.io

5. Reimers, N., & Gurevych, I. (2019).
Sentence-BERT: Sentence Embeddings Using Siamese BERT Networks.
Proceedings of EMNLP-IJCNLP.

6.Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020).
Language Models are Few-Shot Learners.
Advances in Neural Information Processing Systems (NeurIPS).

7.Pinecone Systems Inc. (2024).
Vector Database Documentation.
Available at: https://docs.pinecone.io

8.Sommerville, I. (2016).
Software Engineering (10th Edition).
Pearson Education.